

How does GCC work?

GCC cannot work on its own – it is part program, and partly a wrapper around other programs. To build a working toolchain, we need two packages -- GCC itself and GNU Binutils.

All that GCC does by itself is convert high-level source code into an assembler. GCC includes compilers for C, C++, ADA, Fortran, Objective C, Objective C++ and Java. Barring Java, which is interpreted, all other languages need their programs to be converted into binaries. To explain how this is done, let us do it manually from the command line.

First, create a file called `src.c` with the following contents:

```
#include <stdio.h>

int main()
{
    printf("Hello World!\n");
    return 0;
}
```

Here's a directory listing at the current stage:

```
bg14ina@bg14ina-desktop:~/Desktop/srcs$ ls -l
total 4
-rw-r--r-- 1 bg14ina bg14ina 73 2009-06-07 15:29 src.c
```

First of all, let's compile this source file into the assembler. To do this, we need to execute the following:

```
gcc -S src.c -o src.S
```

...and some command line output:

```
bg14ina@bg14ina-desktop:~/Desktop/srcs$ gcc -S src.c -o src.S
bg14ina@bg14ina-desktop:~/Desktop/srcs$ cat src.S
```

```
    file      "src.c"
    section .rodata
.LC0:
    string   "Hello World!"
    text
.globl main
    type    main, @function
main:
    leal     4(%esp), %ecx
    andl     $-16, %esp
    pushl    4(%ecx)
    pushl    %ebp
    movl     %esp, %ebp
    pushl    %ecx
    subl     $4, %esp
    movl     $LC0, (%esp)
    call     puts
    movl     $0, %eax
    addl     $4, %esp
    popl     %ecx
    popl     %ebp
```

```
    leal     4(%ecx), %esp
    ret
    size     main, .main
    ident    "GCC (Ubuntu 4.3.3-Suburture) 4.3.3"
    section .note.GNU-stack,"",@progbits
bg14ina@bg14ina-desktop:~/Desktop/srcs$ ls -l
total 8
-rw-r--r-- 1 bg14ina bg14ina 73 2009-06-07 15:29 src.c
-rw-r--r-- 1 bg14ina bg14ina 433 2009-06-07 15:40 src.S
```

See that beautifully indented and formatted assembler file? If you're a geek, you could optimise this file in a gazillion ways. I know a little about assemblers, but not enough to actually bring about a 500X increase in execution time. For all purposes, we will now assemble `src.S` into an object file. To do that, execute the following:

```
as -o src.o
```

And the directory listing goes as follows:

```
bg14ina@bg14ina-desktop:~/Desktop/srcs$ ls -l
total 12
-rw-r--r-- 1 bg14ina bg14ina 864 2009-06-07 15:46 a.out
-rw-r--r-- 1 bg14ina bg14ina 73 2009-06-07 15:29 src.c
-rw-r--r-- 1 bg14ina bg14ina 433 2009-06-07 15:40 src.S
```

Notice the *a.out* file? That's our object file. We'll now link that file with the C libraries, to get an executable. The command to do this, however, is huge.

```
ld -eh-frame-hdr -m elf_i386 --hash-style=both --dynamic-linker /lib/ld-linux.so.2 \
-z relro /usr/lib/crti.o /usr/lib/crti.o /usr/lib/gcc/i486-linux-gnu/4.3.3/crtbegin.o \
-L/usr/lib/gcc/i486-linux-gnu/4.3.3 -L/usr/lib/gcc/i486-linux-gnu/4.3.3 -L/usr/lib \
-L/lib /a.out -lgcc -as-needed -lgcc_s -no-as-needed -lc -lgcc -as-needed -lgcc_s \
-no-as-needed /usr/lib/gcc/i486-linux-gnu/4.3.3/crtend.o /usr/lib/crtn.o -o a.exe
```

There's some bad news as well – the linker command is somewhat distro-specific (I'm using Ubuntu Jaunty) and it depends on the GCC version, Binutils version and location of the *crt* object files as well.

Okay, let's test this file:

```
bg14ina@bg14ina-desktop:~/Desktop/srcs$ ls -l
total 24
-rwxr-xr-x 1 bg14ina bg14ina 9143 2009-06-07 16:09 a.exe
-rw-r--r-- 1 bg14ina bg14ina 864 2009-06-07 16:05 a.out
-rw-r--r-- 1 bg14ina bg14ina 73 2009-06-07 16:02 src.c
-rw-r--r-- 1 bg14ina bg14ina 433 2009-06-07 16:04 src.S
bg14ina@bg14ina-desktop:~/Desktop/srcs$ ./a.exe
Hello World!
```

Bingo!

So what now?

Now that we know all about what GCC and the tools from Binutils do, we produce C. GCC converts it to assembly. GAS